

Time Shared Architecture

Lecture 9

Dr. Shoab A. Khan

Time-Shared Designs

- Dedicated Fully Parallel
 - If sampling rate f_s = circuit clock f_{clk}
 - Dedicated Operator for each operation in the algorithm
 - One operator (hardware unit, e.g. adder, multiplier, register) for each operation (e.g addition, multiplication, delay)

- Most designs: time multiplexing
 - **clock frequency ! = sample frequency**
clock frequency = $\frac{\text{sample frequency}}{\text{number of clock cycles available for the job}}$

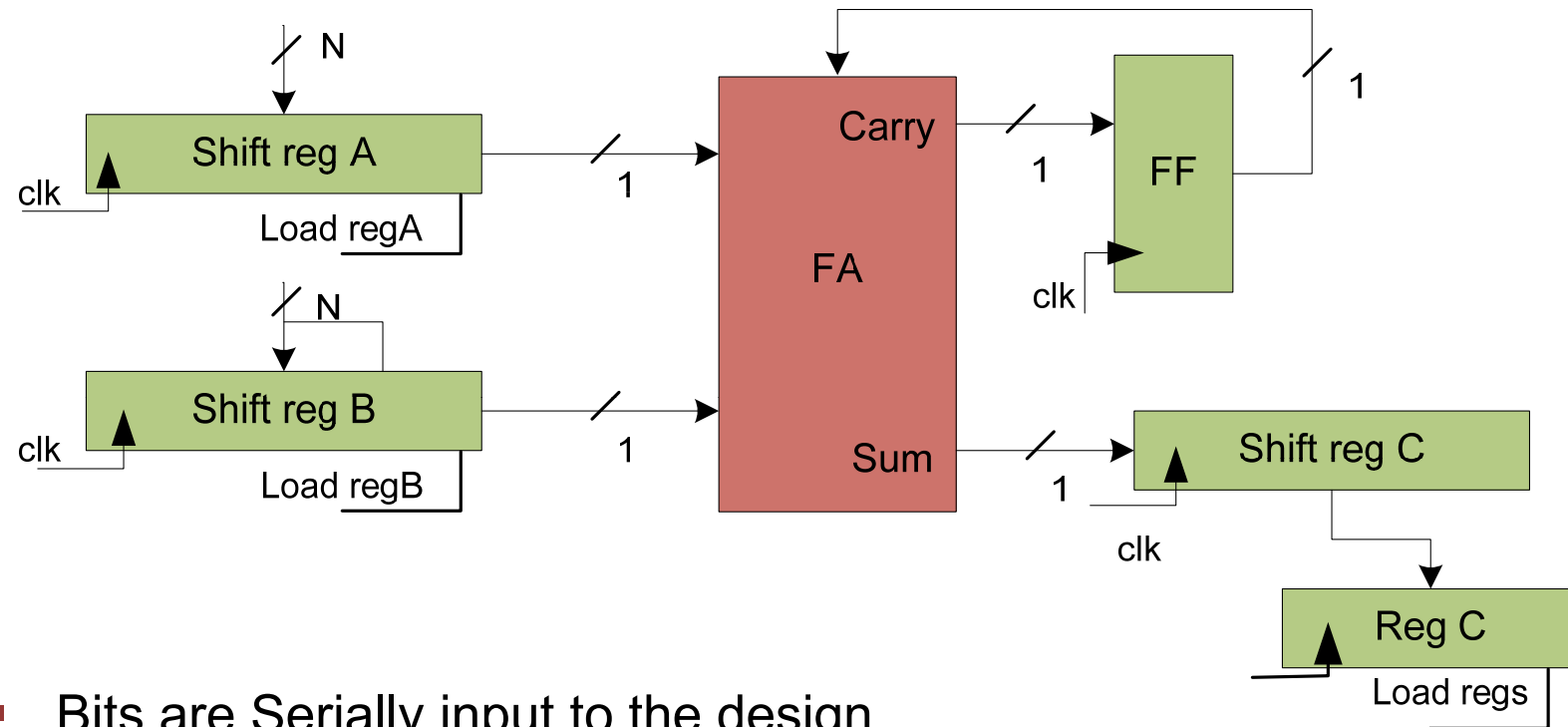
Time Shared Architecture

- Reverse of dedicated architecture
- Less hardware
- Reuse a smaller block to perform complex operations
- Reuse blocks to execute algorithms
- Require controller to schedule operations on the shared HW

Time Shared Architectures

- Bit Serial
- Word Serial
- Sequential
- Sequential Unfolding
- Systolic
- Folded DFG
- Micro-programmed

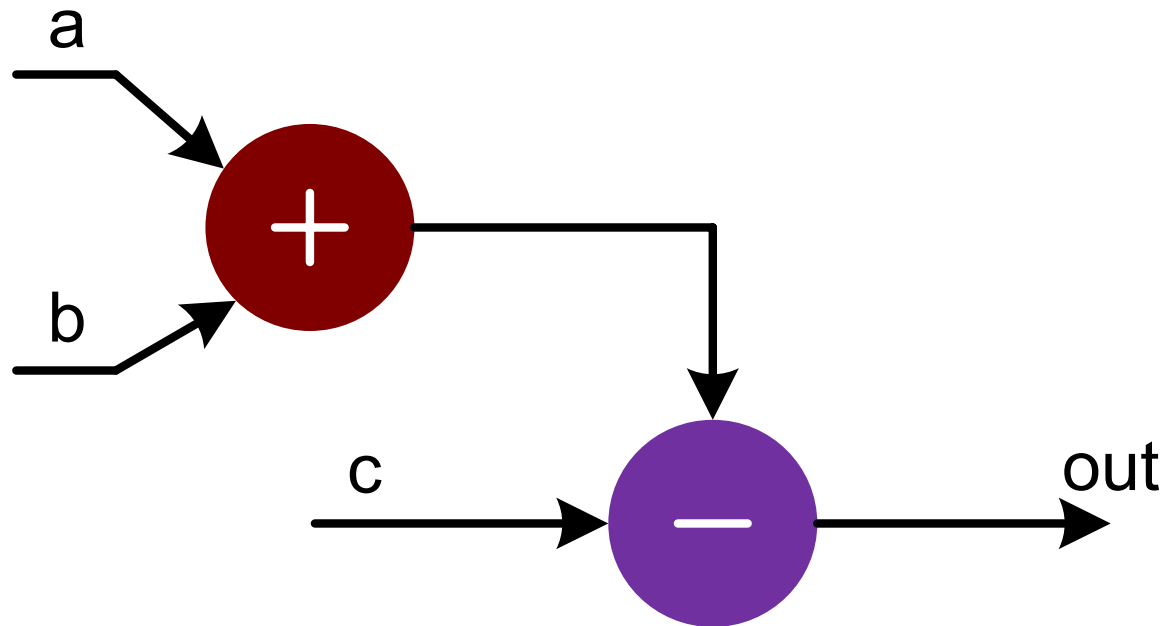
Example Bit Serial: N-bit adder



- Bits are Serially input to the design
- The architecture processes input bit by bit basis
- Bits are out serially

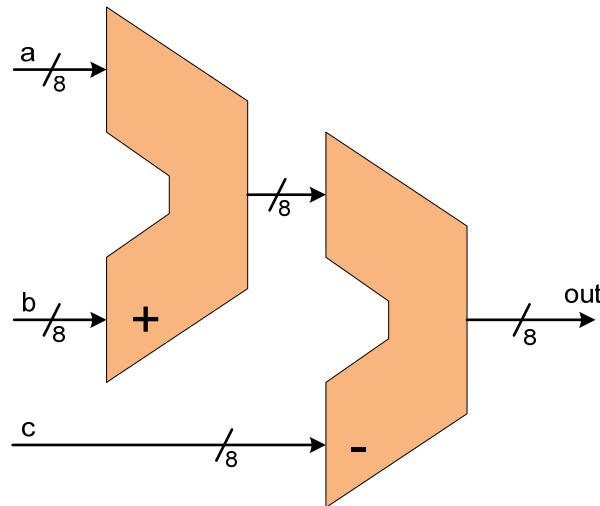
Example: Mapping from Dedicated to Time Shared Architecture

- a, b, c are 8-bit numbers
- The example maps the DFG to dedicated and time shared architecture



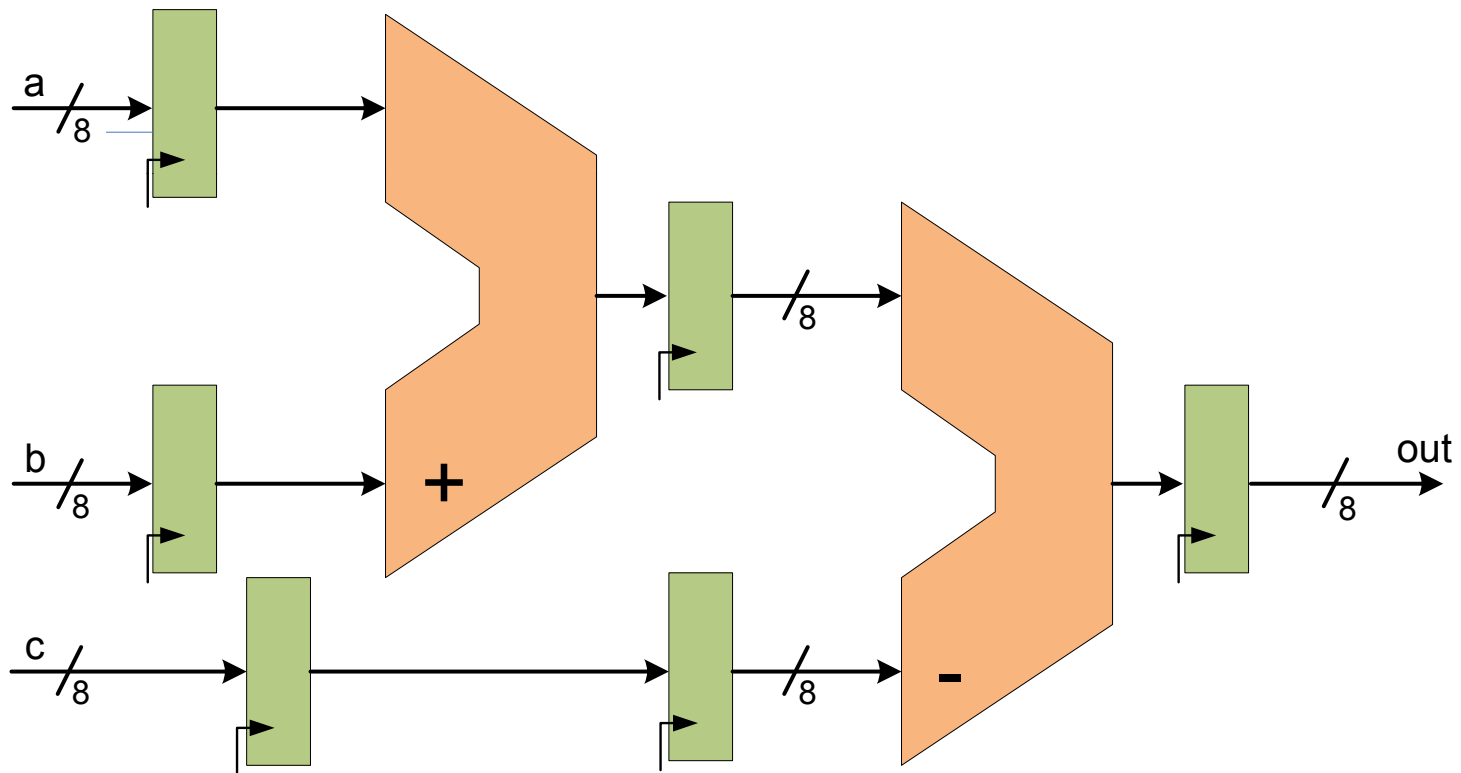
Dedicated Architecture

- Two add bit adder/subtractor for addition and subtraction
- No pipelining
- Need to add pipelining for better performance



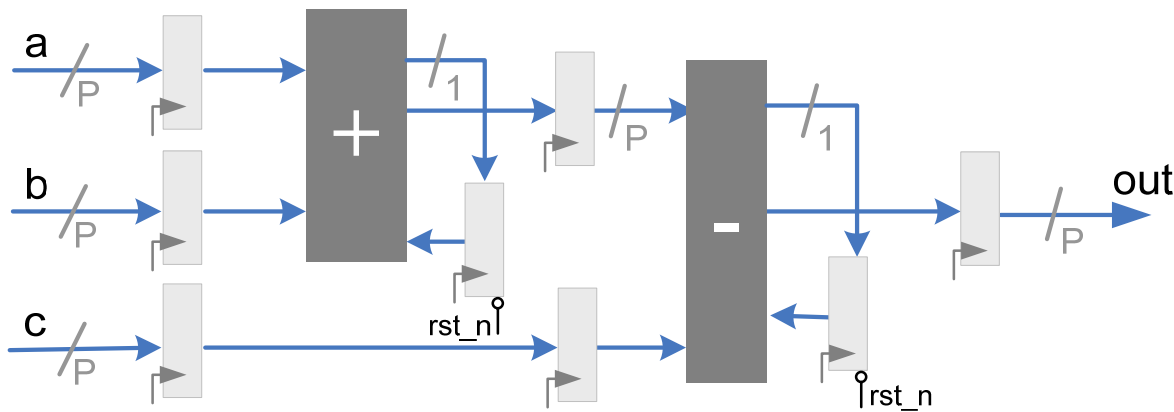
Pipeline Architecture

- Remember data coherency is important



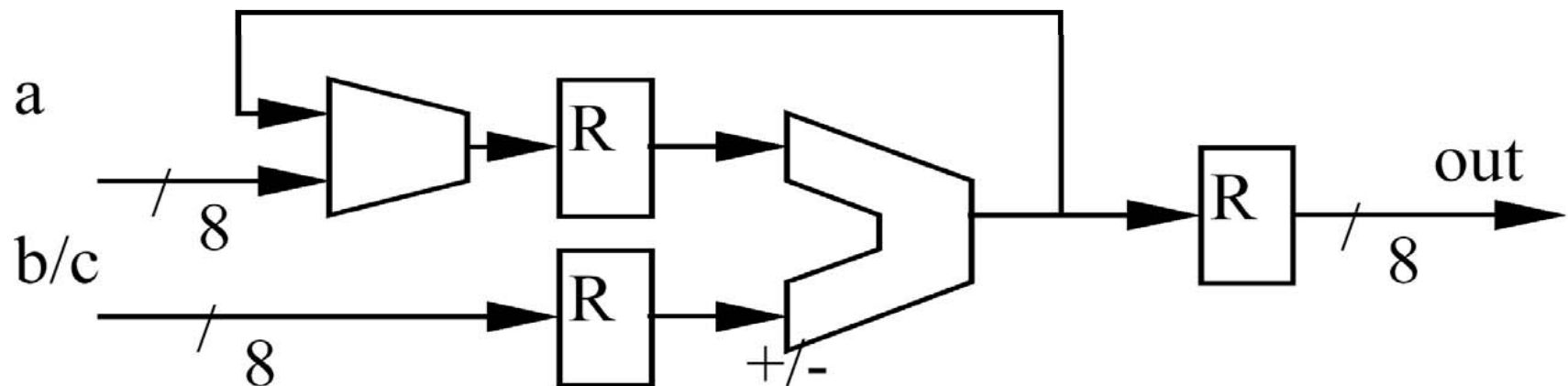
If f_c is N times $f_s \rightarrow$ bit-serial

- If N=8 cycles are available: Bit Serial
 - Bit-serial data in
 - Two FA for 1-bit addition and subtraction
 - Flip flops for pipelining
- Word Serial
 - If N=8, 4, 2 the number of bits P=1, 2, 4



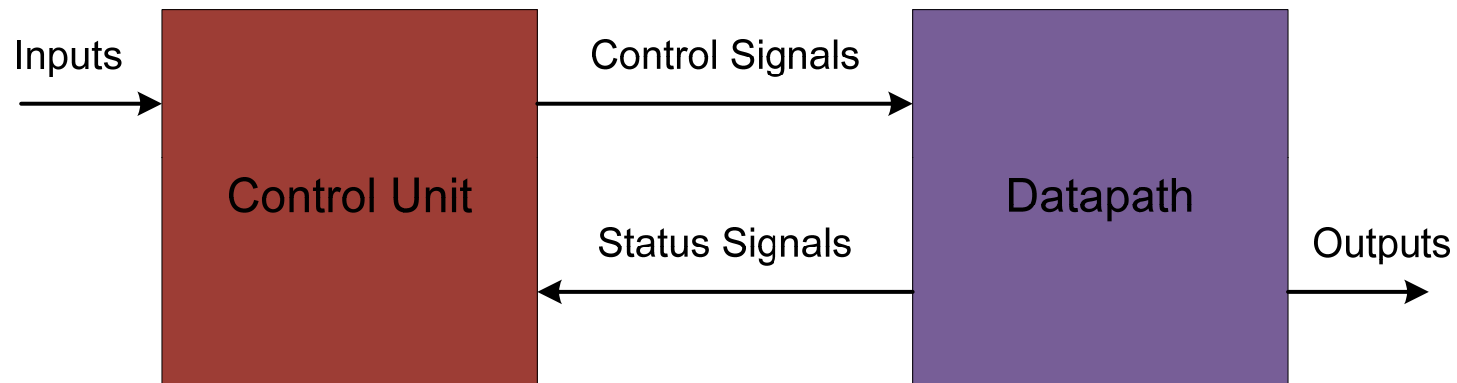
If f_c is twice of $f_s \rightarrow$ Folded Design

- If $N=8$ cycles are available: Bit Serial
- Time shared architecture
- One adder/subtractor is used for addition and subtraction to compute $a+b-c$
- Two cycles to execute the DFG



Sequencing and Control

- Digital Systems can be partitioned into two portions
- Datapath and Control Unit



Puppeteer Controller who pulls the strings

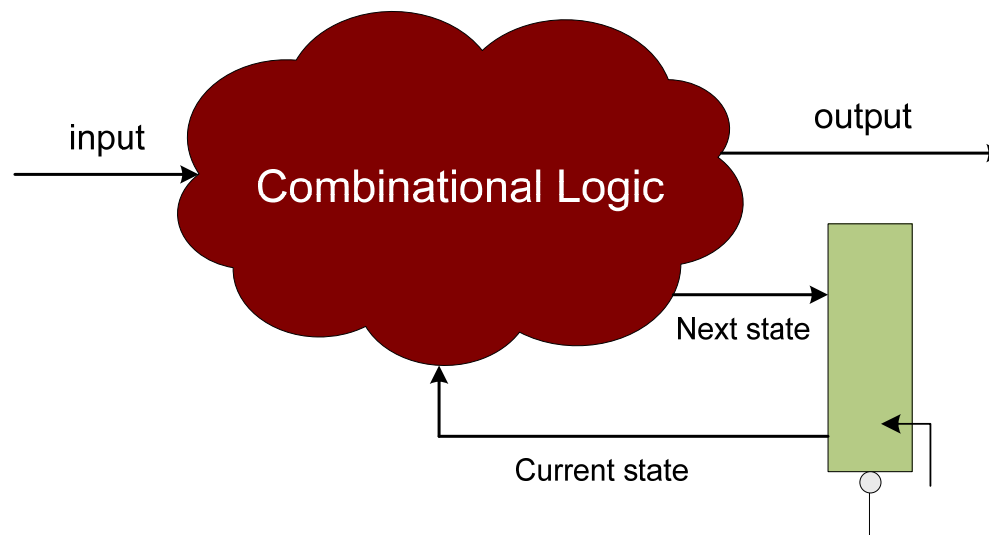
- Selects the operation
- Determines the sequence (based on status and input signals)

Datapath Puppet

- Registers, MUXes, ALU, Multipliers, Shifters, Comb, Circuits and buses
- Implements operations under control of the control signals

State machine

- A machine activity is usually consists of a synchronous sequence of operations on the registers of its datapaths, under the direction of a controlling state machine



ASM Implementation

- Two general implementation methods
 - Hardwired
 - Micro-programmed

- Two representation
 - State Diagram
 - Algorithmic State Machine

- Two Types of State Machines
 - Mealy State Machine
 - Moor State Machine

A Finite State Machine (FSM) Controller

- Controller are of two types:
 - Hardwired Finite State Machine based controller
 - Microprogramm Architecture based controller
- A FSM is a sequential system with N flip-Flops and has 2^N possible states, so the number of possible states is FINITE
- FSM can be described using a Bubble Diagrams (State Diagram) or Algorithmic State Machine Charts (ASM)

A Finite State Machine (FSM) Controller

- Controller are of two types:
 - Hardwired Finite State Machine based controller
 - Microprogramm Architecture based controller
- A FSM is a sequential system with N flip-Flops and has 2^N possible states, so the number of possible states is FINITE
- FSM can be described using a Bubble Diagrams (State Diagram) or Algorithmic State Machine Charts (ASM)

FSM Comparison

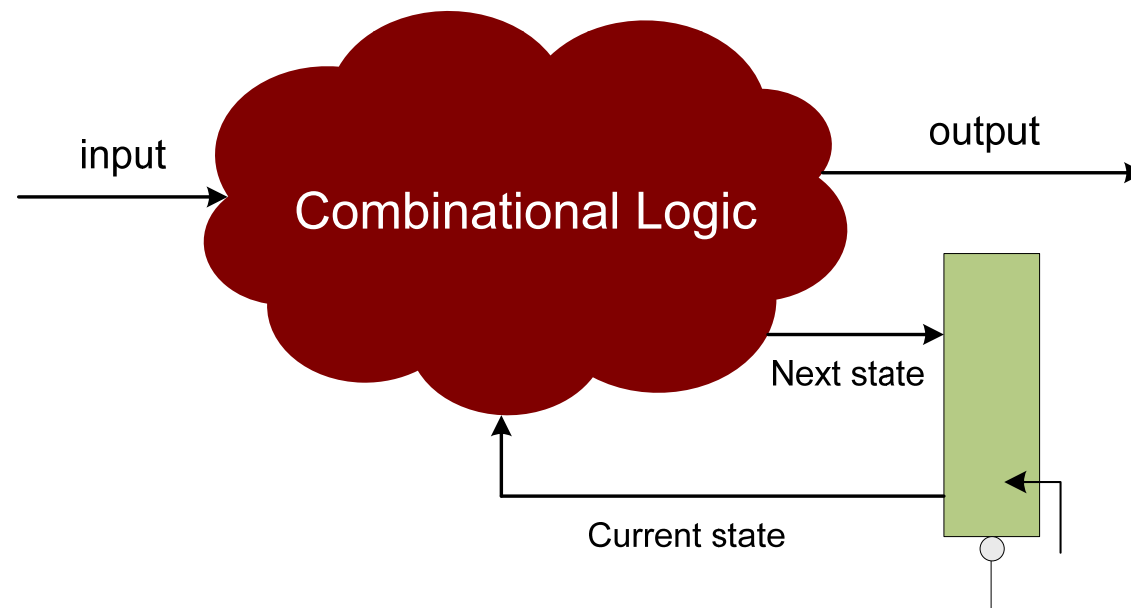
Moore Machine

- output function only of present state
- maybe more state
- synchronous outputs
 - no glitching
 - one cycle “delay”
 - full cycle of stable output

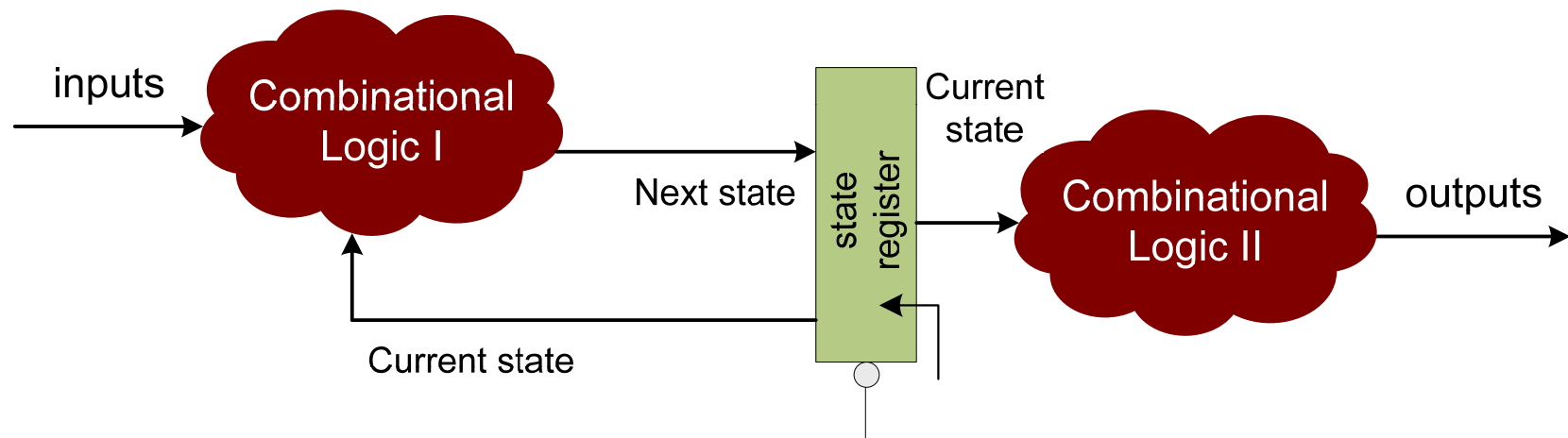
Mealy Machine

- output function of both present states & input
- maybe fewer states
- asynchronous outputs
 - if input glitches, so does output
 - output immediately available
 - output may not be stable long enough to be useful

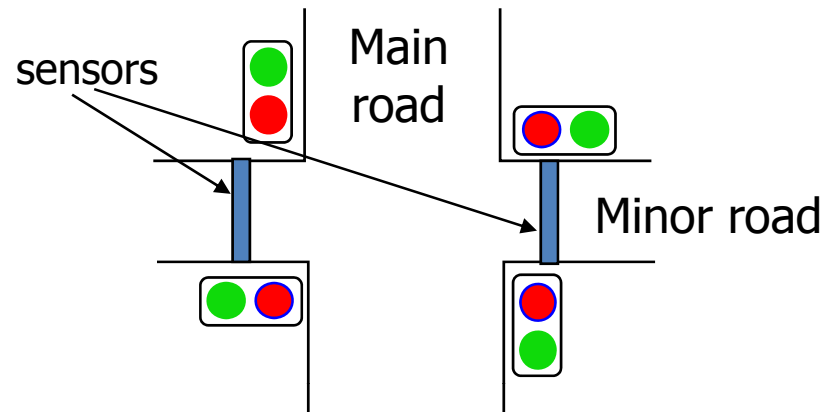
The composition of Mealy machine implementation of an FSM



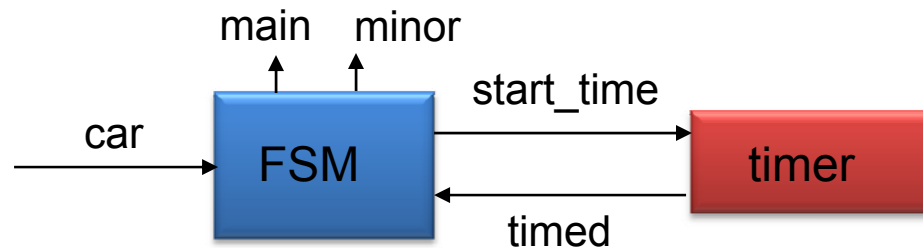
The components in Moore machine implementation of a state machine based design



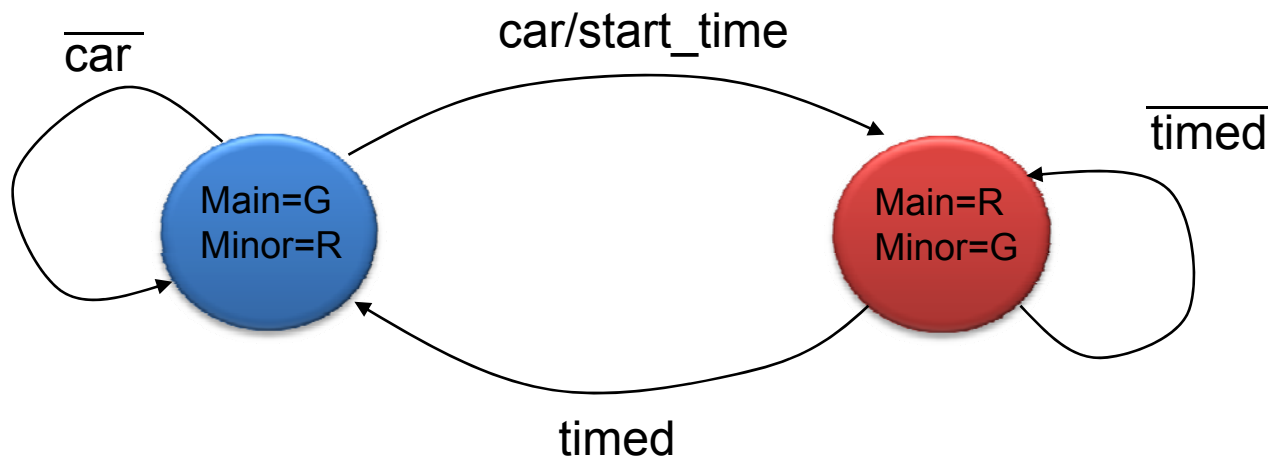
Example: Simple traffic signal controller



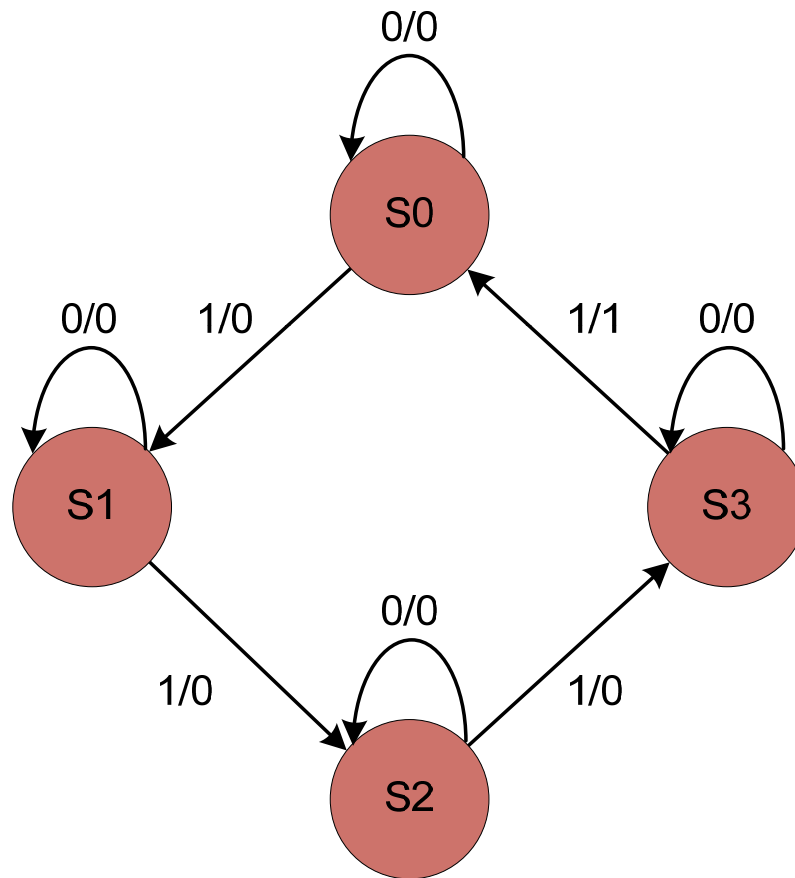
- Main road has normally a green light, and minor road a red light
- If a car is detected on minor road (sensor), semaphores change values, a timer is started which asserts a signal 'TIMED' at the end of counting
- When TIMED is asserted, the semaphores go back to default values



State diagram



State diagram



State Encoding: One-Hot versus Binary Coded Finite-State Assignment

<pre>parameter [1:0] S0 = 2'd00, S1 = 2'd01, S2 = 2'd10, S3 = 2'd11;</pre>	<pre>parameter [3:0] S0 = 4'b0001, S1 = 4'b0010, S2 = 4'b0100, S3 = 4'b1000;</pre>	<pre>parameter [2:0] S0 = 4'b000, S1 = 4'b001, S2 = 4'b010, S3 = 4'b100;</pre>	<pre>parameter [2:0] S0 = 4'b000, S1 = 4'b001, S2 = 4'b011, S3 = 4'b010;</pre>
(a)	(b)	(c)	(d)
State Encoding Techniques (a) Binary (b) One-Hot (c) Almost One-Hot (d) Gray			

Guideline for Coding State Machines

- Separate the state machine HDL description into two processes, one for the combinational logic and one for the sequential logic
- Use ``define`/parameter statements to define a state vector.
- Keep FSM logic and non-FSM logic in separate modules.

Translating FSMs into Verilog HDL: Combinational Part

```
always @(current_state or in)
```

```
begin
```

```
case (current_state)
```

```
  S0:
```

```
  begin
```

```
    if (in)
```

```
      begin next_state = S1; out = 1'b0; end
```

```
    else
```

```
      begin next_state = S0; out = 1'b0; end
```

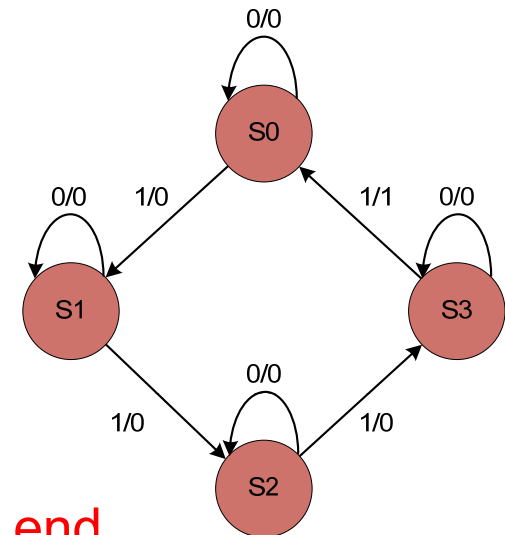
```
  end
```

```
  S1:
```

```
  begin...
```

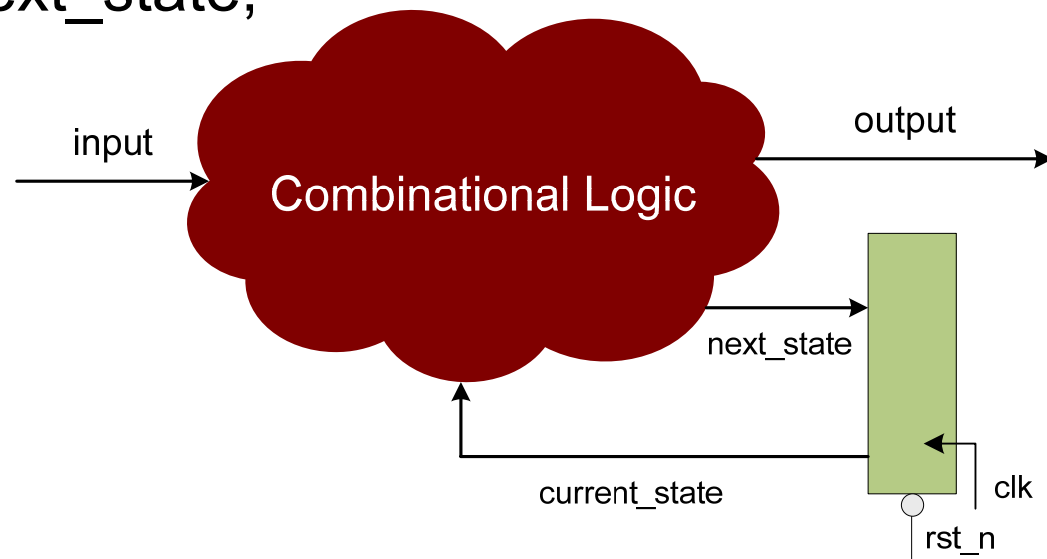
```
  end
```

```
endcase
```



Sequential Part

```
always @(posedge clk or negedge res_n)
begin
if (!res_n)
    current_state <= S0;
else
    current_state <= next_state;
end
```



Why state diagrams are not enough

- Not flexible enough for describing very complex finite state machines
- Not suitable for gradual refinement of finite state machine
- Do not obviously describe an algorithm: that is, well specified
- Gradual shift towards program-like representations:
 - Algorithmic State Machine (ASM) Notation
 - Hardware Description Languages (e.g., Verilog, VHDL)

Algorithmic State Machine

- A flowchart-like graphical notation that describes the cycle-by-cycle operations of an algorithm
- Each step takes one clock cycle
- Composed of rectangles, diamonds, ovals, and arrows interconnecting them
- Moore machines do not have ovals
- Mealy machines contain ovals
- Describes behavior rather than structure
- Provides a mechanism for performing systematic step-by-step design
- Can be directly translated to Verilog code
- Used to design synchronous sequential circuits

Algorithmic State Chart (ASM)

- An ASM chart is used to describe FSM behavior
 - Only three action signals can appear within an ASM chart:



- **State box.** Each box represents a state. Outputs within a state box is an UNCONDITIONAL output (always asserted in this state)

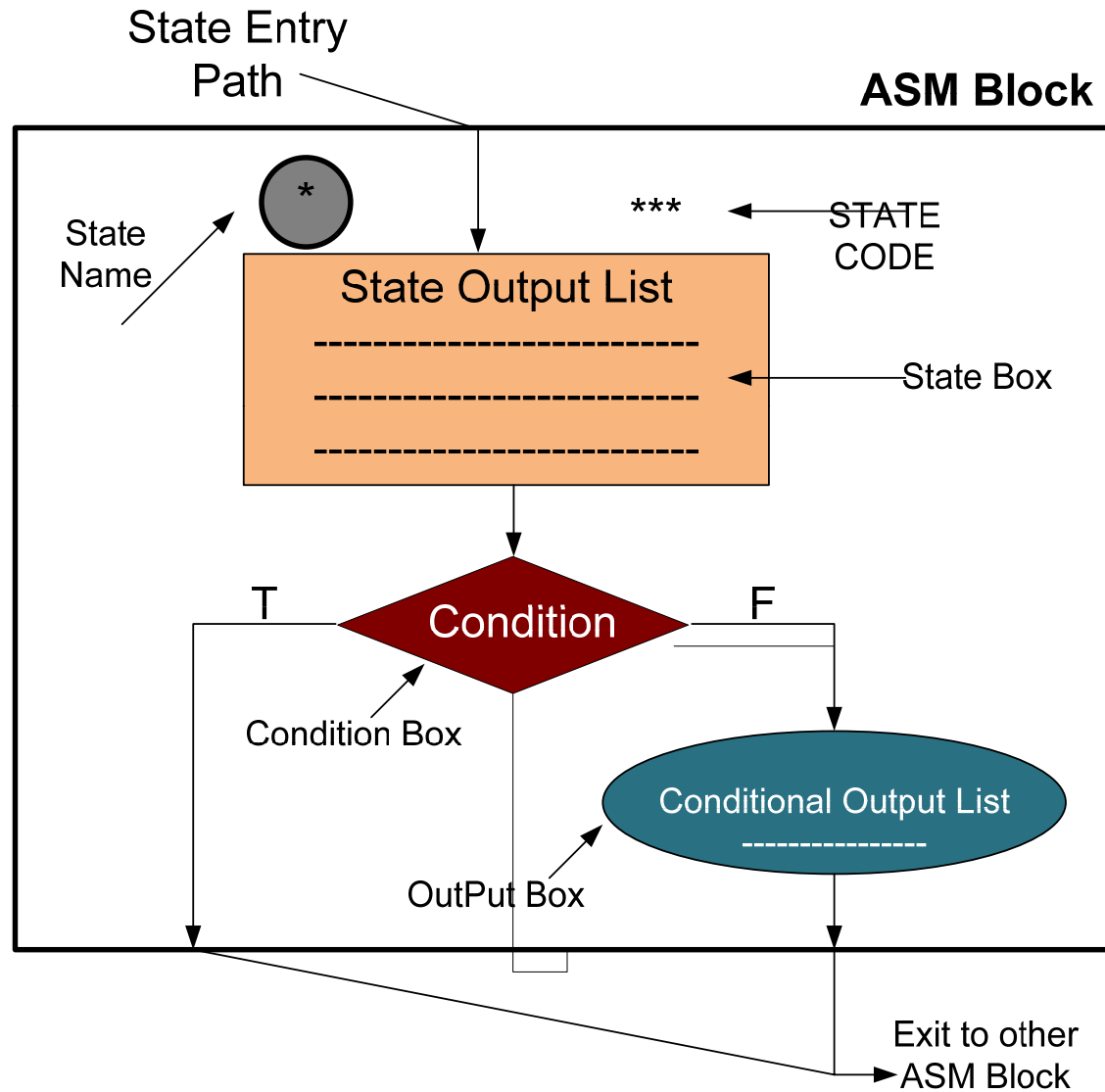


- **Decision box.** A condition in this box will decide next state condition

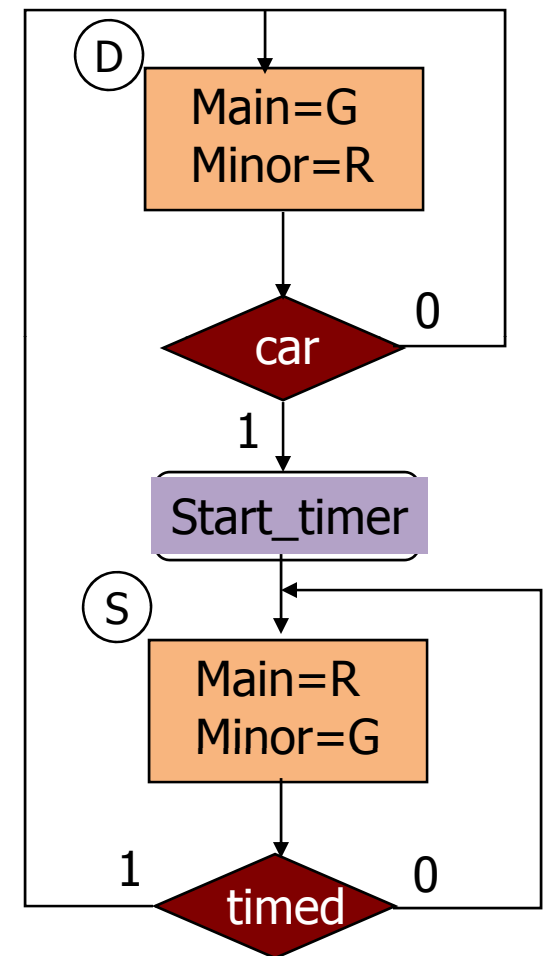
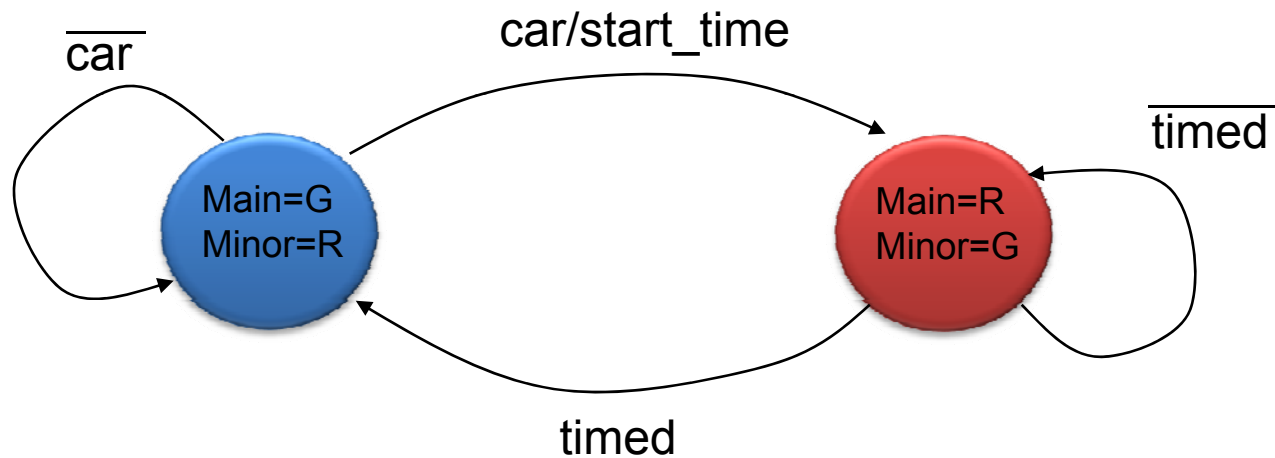


- **Conditional output box.** If present, will always follow a decision box; output within it is conditional.

ASM Block

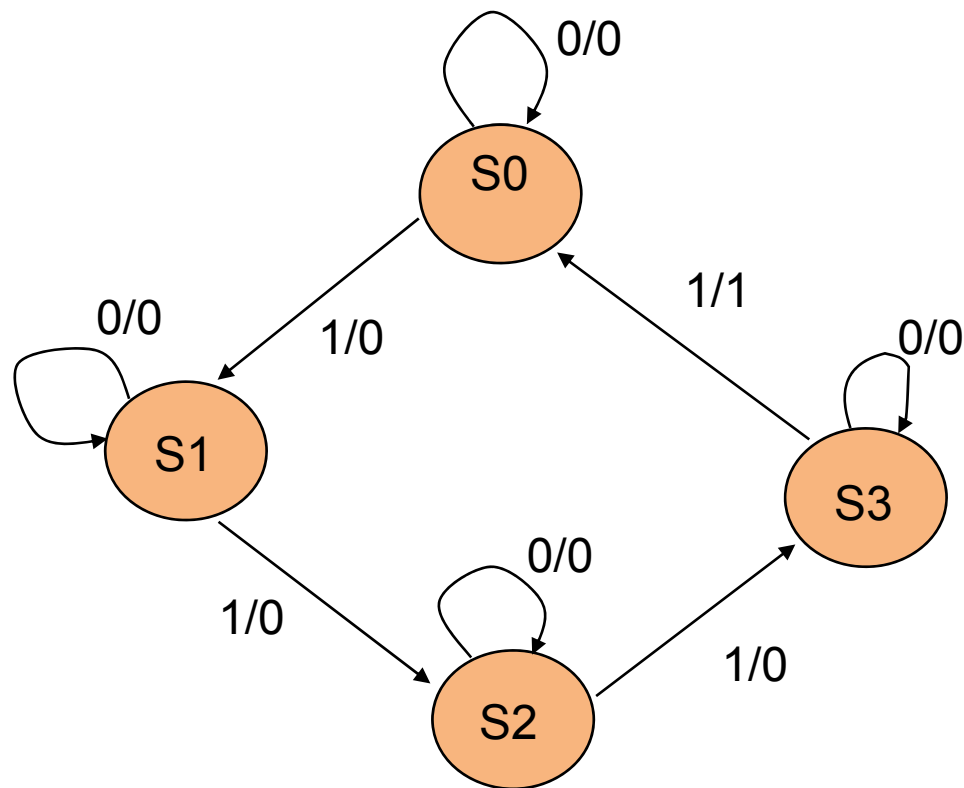


Algorithmic state machine (ASM)

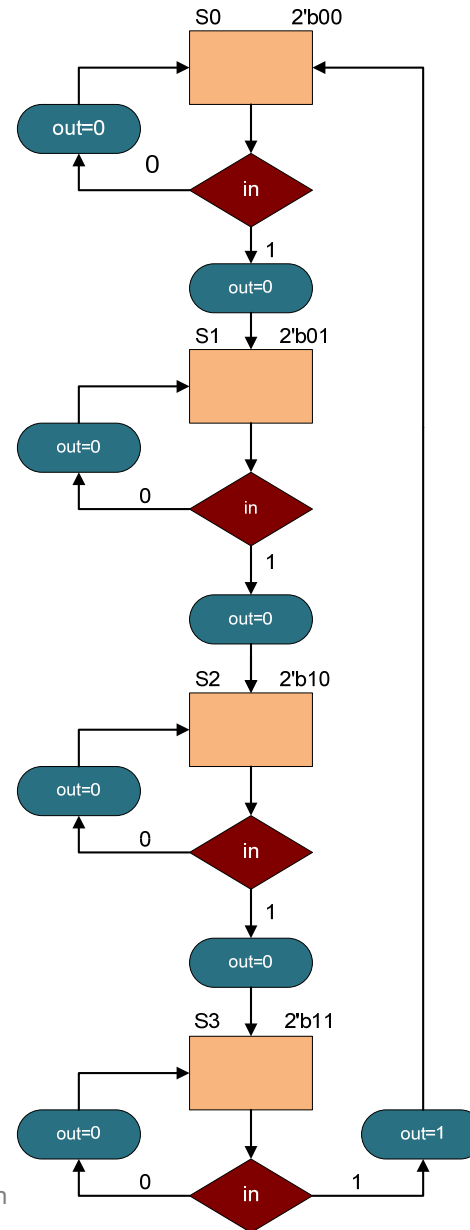
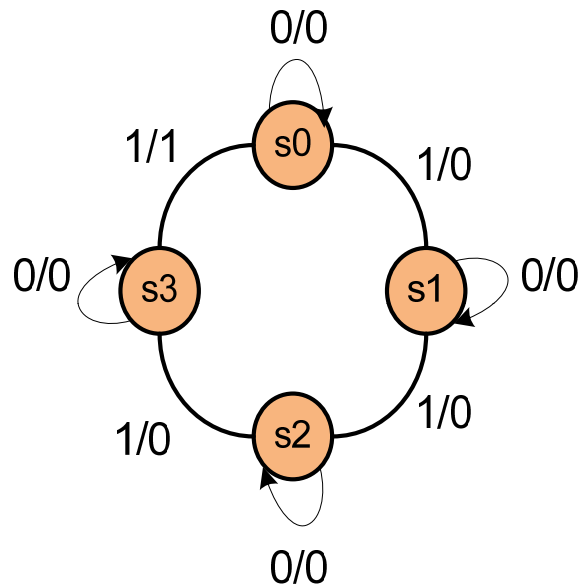


State Diagram

- Generating 1 at the output after counting four number of 1's on a serial interface

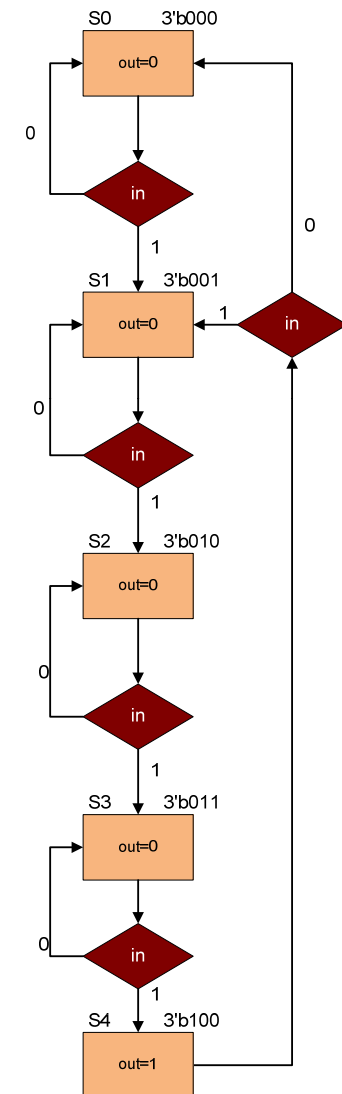


Example: Mealy Machine- State Diagram, ASM Representation



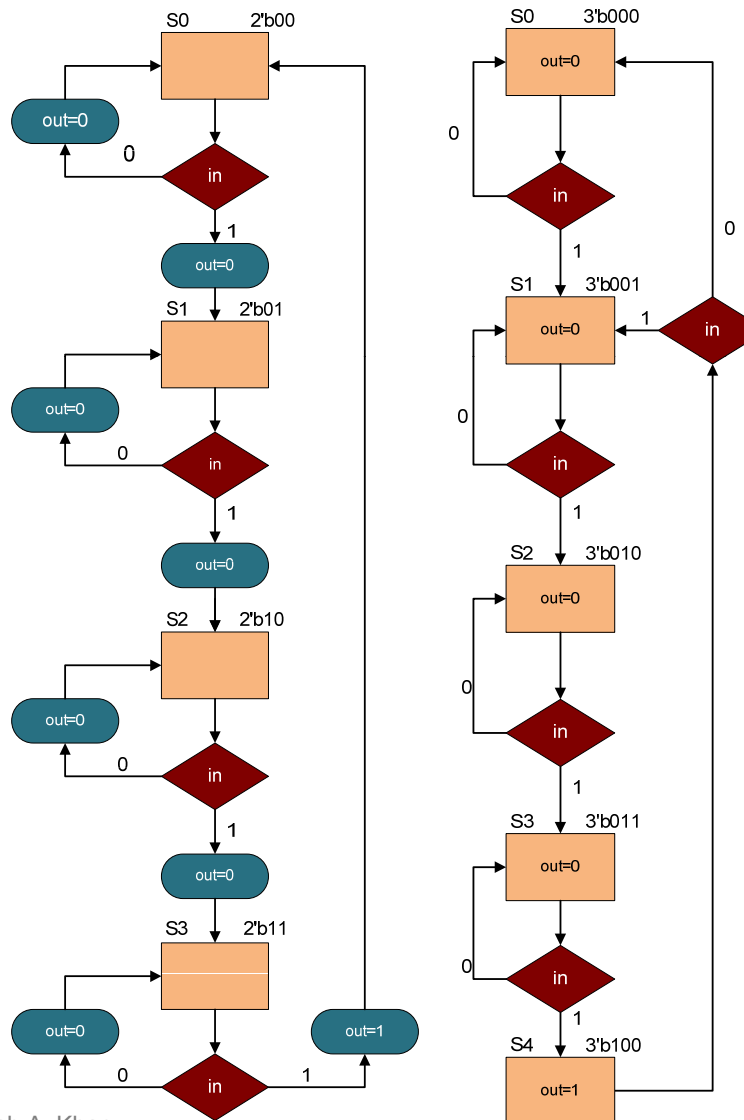
Example: Mealy Machine- State Diagram, ASM Representation

- No oval, means no conditional output
- More states
- Stable output for one complete clock cycle



Example: Mealy Machine- State Diagram, ASM Representation

- Mealy-Moore side by side
- Design independent
- Simple tradeoff

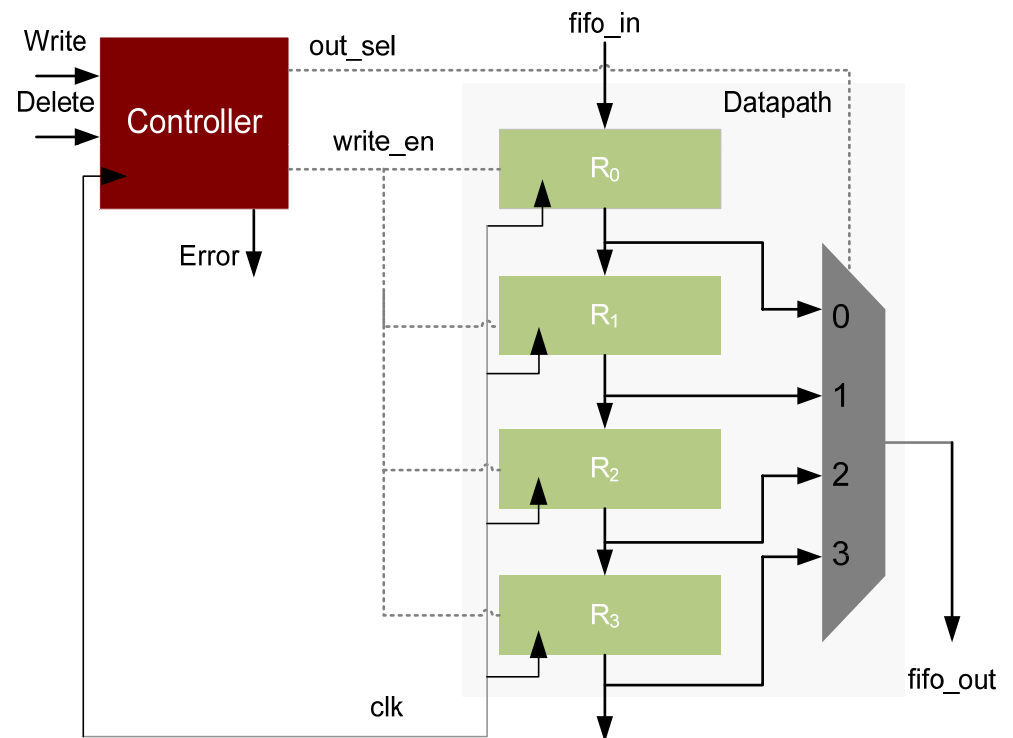


Design Example: 4 Entry FIFO

1. Design a first-in, first-out (FIFO) queue that consists of four registers R0, R1, R2, and R3
2. Write and Delete are the two operations on the queue
3. Write moves data from the `fifo_in` to R0 that is the tail of the queue
4. Delete deletes the first entry at the head of the queue
5. The head of the queue is available on the `fifo_out`
6. Writing into a full queue or deletion from an empty queue causes an ERROR condition
7. Assertion of Write and Delete at the same time also causes an ERROR condition

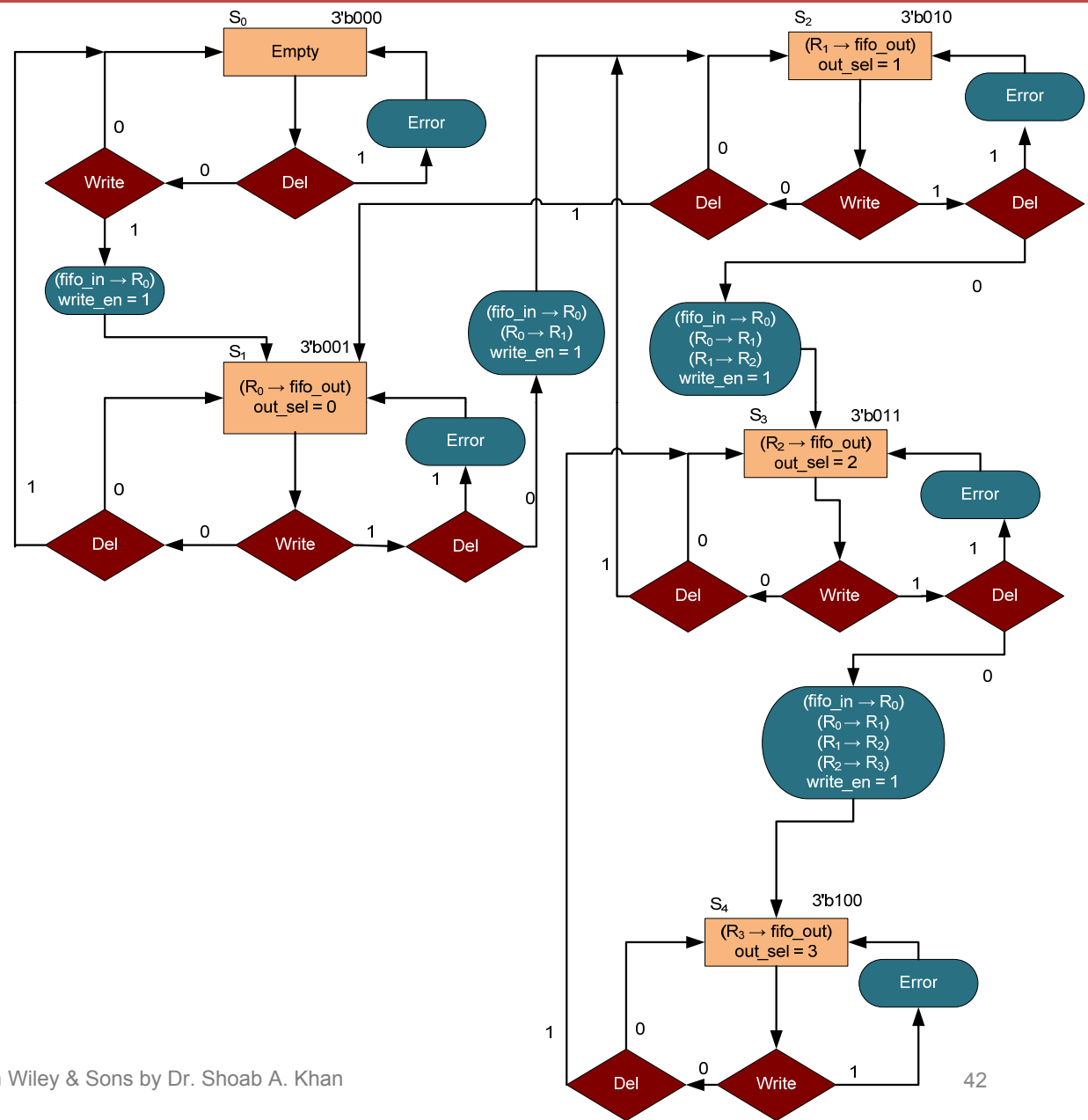
FIFO Design

- Design consists of datapath and controller
- Datapath
 - Four registers
 - Shift registers
 - One 4:1 MUX
- Controller
 - Input signals
 - Write
 - Delete
 - Output signals
 - out_sel
 - write_en
 - Error



Controller

- FSM based design
 - Mealy machine
- Five states
 - Idle
 - One entry
 - Two entries
 - Three entries
 - Full



```

// Combinational part only for S0 and default state is given
always @(*)
begin
    next_state=0;
    case(current_state)

S0:
begin
    if(!Del&& Write)
    begin
        next_state = S1;
        write_en = 1'b1;
        Error= 1'b0;
        out_sel = 0;
    end
    else if(Del)
    begin
        next_state=S0;
        write_en =1'b0;
        Error = 1'b1;
        out_sel=0;
    end
    else
    begin
        next_state=S0;
        write_en=1'b0;
        out_sel = 1'b0;
    end
    // Similarly, rest of the states are coded //
    default:
    begin
        next_state=S0;
        write_en = 1'b0;
        Error = 1'b0;
        out_sel =0;
    end
    endcase
end

// Sequential part
always @(posedge clk or negedge rst_n)
if(!rst_n)
    current_sate <= #1 S0;
else
    current_state <= #1 next_sate;

```